Knowledge Database Module - Sprint 1 (Prototype) Specification

Target stack: **Node.js / NestJS** (API) + **Angular** (Frontend) + **PostgreSQL** (DB).

Document date: 2026-03-02 (Europe/Lisbon)

1. Goal and Scope

Sprint 1 delivers a **functional prototype** of the Knowledge Database (Lessons Learned) module: users can submit an item, see it in a listing, and open a detail view. The prototype also includes a read-only “Documents Used by Project” view and basic statistics.

In scope (Sprint 1)

- DB schema + migrations
- Seed data (types, processes, plants, sample items, sample projects)
- NestJS API with Create + Read endpoints (no edit/delete yet)
- Angular pages: Listing, Submit, Detail, Documents Used, Stats
- Basic filtering on Listing (simple query params)
- File attachments: store metadata + upload to local storage (or stub storage)

Out of scope (explicitly NOT in Sprint 1)

- Approval workflow (validate/approve/reject), notifications
- Advanced RBAC/permissions by role and process (only minimal guards/menu visibility)
- Editing, deactivation, version history, audit trail
- Full project lifecycle integration

2. User Journey (Happy Path)

1. User opens **Knowledge Listing** (default page).
2. User clicks **+ Submit New Item**.
3. User fills the form and uploads an optional document/images, then submits.
4. System creates the item with status **PENDING** and redirects to **Detail**.
5. Item appears in Listing with a status badge.
6. User can open Stats and Documents Used pages (read-only).

3. Information Architecture

Navigation structure:

Knowledge DB	Pages
Listing (Home)	Table + filters, opens Detail
Submit New Item	Form; creates item; redirect to Detail
Documents Used	Select project; list linked items (read-only)
Stats	Counts by Type/Process/Plant (read-only)

4. Data Model (PostgreSQL)

Use UUID primary keys. Use snake_case table names. Use soft deletion flags where applicable.

4.1 knowledge_item

Column	Type	Notes	Required
id	uuid	PK	Y
title	text	Main title shown in listing/detail	Y
designation	text	Short designation; optional	N
item_date	date	Business date of the item	Y
owner	text	Owner name/identifier (string for prototype)	Y
author	text	Author name/identifier (string for prototype)	Y
type_id	uuid	FK -> master_type(id)	Y
project_code	text	Project/program identifier (string for prototype)	Y
plant_code	text	Plant identifier (string for prototype)	Y
visibility_status	text	Enum: PENDING|VISIBLE|NOT_VISIBLE. Default PENDING	Y
is_active	boolean	Default true	Y
created_at	timestamptz	Default now()	Y

4.2 master_type

Column	Type	Notes	Required
id	uuid	PK	Y
code	text	Unique code (e.g., GUIDELINE)	Y
label	text	Display name	Y
is_active	boolean	Default true	Y

4.3 master_process

Column	Type	Notes	Required
id	uuid	PK	Y
code	text	Unique code (e.g., CHROME)	Y
label	text	Display name	Y
is_active	boolean	Default true	Y

4.4 knowledge_item_process (many-to-many)

Column	Type	Notes	Required

knowledge_item_id	uuid	FK -> knowledge_item(id)	Y
process_id	uuid	FK -> master_process(id)	Y

4.5 knowledge_item_file

Column	Type	Notes	Required
id	uuid	PK	Y
knowledge_item_id	uuid	FK -> knowledge_item(id)	Y
file_kind	text	Enum: DOCUMENT|IMAGE	Y
filename_original	text	Original name	Y
storage_path	text	Local path or URL (prototype)	Y
uploaded_at	timestamptz	Default now()	Y

4.6 project_knowledge_item

Column	Type	Notes	Required
project_code	text	Project identifier	Y
knowledge_item_id	uuid	FK -> knowledge_item(id)	Y
added_at	timestamptz	Default now()	Y

Seed data requirements

- At least 5 Types and 5 Processes.
- At least 2 Plants and 2 Projects.
- At least 10 knowledge items; at least 1 item linked to multiple processes; at least 1 item with attachments.
- Optionally link 2-3 items to each project in project_knowledge_item.

5. API Specification (NestJS)

Base URL: /api. Use JSON. Use class-validator DTOs. Use Swagger decorators for documentation.

5.1 Endpoints

Method	Path	Description
GET	/api/knowledge	List items with optional filters (query params)
GET	/api/knowledge/:id	Get item detail (including processes + files)
POST	/api/knowledge	Create item (multipart/form-data for files)
GET	/api/projects/:projectCode/documents-used	List items linked to project (read-only)
GET	/api/knowledge/stats	Counts by type/process/plant
GET	/api/master/types	List active types (for dropdowns)
GET	/api/master/processes	List active processes (for dropdowns)

5.2 Listing filters (GET /api/knowledge)

- typeId (uuid)
- processId (uuid)
- projectCode (string)
- owner (string)
- author (string)
- plantCode (string)
- dateFrom (YYYY-MM-DD)
- dateTo (YYYY-MM-DD)
- page, pageSize (optional)

5.3 DTOs (prototype-level)

- CreateKnowledgeItemDto: title, designation?, itemDate, owner, author, typeId, projectCode, plantCode, processIds[]
- File uploads: document (single), images (multiple)
- KnowledgeItemListDto: id, title, designation, typeLabel, processes[], owner, projectCode, itemDate, visibilityStatus
- KnowledgeItemDetailDto: all fields + files[]

5.4 Implementation notes

- Use TypeORM or Prisma (choose one; Prisma recommended for speed).
- Use Multer for multipart uploads; store files under ./storage/knowledge/{itemId}/.
- Store only file metadata in DB; do not implement virus scan or cloud storage in Sprint 1.
- Set visibility_status to PENDING on create.

6. Frontend Specification (Angular)

Use Angular routing. Use a shared layout component for the module (title, breadcrumb, toolbar).

6.1 Pages and components

Route	Component	Key UI elements
/knowledge	KnowledgeListPage	Filters panel + table; opens detail; button to submit
/knowledge/new	KnowledgeSubmitPage	Form with dropdowns and uploads; submit -> redirect to detail
/knowledge/:id	KnowledgeDetailPage	Metadata + files list
/documents-used	DocumentsUsedPage	Project selector + read-only table
/stats	KnowledgeStatsPage	3 cards/tables with aggregates

6.2 Shared services

- KnowledgeApiService: list(filters), getById(id), create(formData), stats(), documentsUsed(projectCode)
- MasterDataService: getTypes(), getProcesses()
- Optional: Toast/Notification service for success messages

6.3 UI fields mapping (Submit form)

UI Field	Backend field
Title *	title
Designation	designation
Date *	itemDate
Owner *	owner
Author *	author
Type * (dropdown)	typeId
Project *	projectCode
Plant *	plantCode
Process (multi-select)	processIds[]
Upload Document	document file
Upload Images	images[] files

7. Acceptance Criteria (Sprint 1)

- User can create a knowledge item; status defaults to PENDING.
- Created item appears in Listing and can be opened in Detail.
- Listing filters work at least for Type and Process (others optional).
- Stats endpoint returns correct counts and UI displays them.
- Documents Used page loads items for a selected project.
- Seed script creates data without errors.
- No runtime errors in API or UI; basic form validation for required fields.

8. Implementation Checklist

- NestJS project scaffold with modules: knowledge, master-data, projects, files/storage
- DB migrations + seed runner
- Swagger available at /api/docs
- Angular module + routing; reusable components for table, badges, filters
- Environment config (API base URL) and CORS enabled in NestJS